Spatio-Temporal 5G QoS Forecasting from Real Network Measurements, with Real-Time Monitoring

Sabrine Assili
National School of Electronics and Telecommunications of Sfax (ENET'Com), Tunisia
Source code: github.com/Sabrine74/5g-spatiotemporal-qos-forecasting
LinkedIn: linkedin.com/in/sabrineassili74

***Abstract*— Applications such as adaptive video streaming and connected vehicles need to know the quality of service (QoS) a mobile network will deliver *in the next seconds*, not only the current one. We study short-term downlink throughput forecasting on real measurements from a commercial 5G network (21 traces, 45,768 seconds, driving and static), using four groups of information available at time t: throughput history, radio channel indicators, mobility (speed, Doppler shift, handovers) and space (position and a space-time throughput prior learned from other drives). With a leakage-free protocol that holds out entire traces, a gradient-boosted model (LightGBM) beats the persistence baseline in every fold, reducing the error of the mean throughput over the next 1, 3 and 5 s by 3, 16 and 25 % (MAE) and 15, 25 and 30 % (RMSE). An ablation study shows that, on average, history carries almost all the predictive information; radio and mobility features help only where history fails, right after handovers, where they turn a model worse than persistence into one 3 % better. The space-time prior does not improve accuracy on this dataset, and normalised error does not grow with speed, which we explain with channel coherence time. Conformalised quantile regression gives 80 % prediction intervals with 78 % empirical coverage on unseen drives (72 % without calibration). In a trace-driven video-streaming simulation, using the forecast instead of the classic harmonic-mean estimator raises the quality of experience on 19 of 21 traces. Finally, we package the model as a real-time monitoring system (FastAPI, InfluxDB, Grafana) with predictive alerts.**

***Index Terms*— 5G, QoS prediction, predictive QoS, throughput forecasting, spatio-temporal data, handover, Doppler, conformal prediction, adaptive video streaming, Grafana.**

I. Introduction

The throughput of a mobile connection changes from second to second: the radio channel fades, the user moves between cells, and other users share the same cell. Many applications would behave better if they knew this throughput *a few seconds in advance*. A video player could pick the right quality before a drop instead of stalling; a connected car could lower the resolution of its sensor stream before entering a coverage hole; a remote driving session could slow down the vehicle in time. This idea is called *predictive QoS*: the automotive industry has identified it as a key enabler of 5G-connected vehicles [1], and 3GPP has standardised a network data analytics function (NWDAF) that exposes QoS predictions to applications [2].

Forecasting throughput from measurements is an old problem in 4G [3] and has been studied for commercial mmWave 5G [4]. Three questions remain open for a practitioner who wants to build such a system on real data:

- **Q1 – Accuracy.** Can a learned model beat the trivial forecast "the next seconds will look like now" (*persistence*) on drives it has never seen?
- **Q2 – Information.** What helps beyond past throughput: radio indicators, mobility (speed, Doppler, handovers), or the *location* of the user, i.e. a spatio-temporal ("4D": position + time) view of QoS?
- **Q3 – Usefulness.** Can the forecast come with a reliable uncertainty, and does it improve a real application?

This work answers these questions on a public dataset collected on a commercial 5G network [5]. Its **contributions** are:

1. a leakage-free forecasting protocol on real 5G traces (whole drives held out, validity mask on gaps), with naive baselines that a model must beat;
2. an ablation of four feature groups, including physically motivated mobility features (Doppler shift, handovers) and a space-time throughput prior evaluated at the position the vehicle will reach in h seconds;
3. calibrated 80 % prediction intervals by conformalised quantile regression;
4. a trace-driven video-streaming evaluation of the forecast's practical value;
5. an open-source real-time monitoring stack (FastAPI, InfluxDB, Grafana) with predictive alerts.

Section II presents the dataset, Section III the method and Section IV the real-time system. Section V reports the results, Section VI discusses them and Section VII concludes.

II. Dataset

A. Source

We use the 5G production dataset of Raca *et al.* [5], collected in Cork, Ireland, between December 2019 and February 2020 on the network of a major Irish operator. A phone running the G-NetTrack Pro application logged, about once per second, the GPS position and speed, the serving cell, the radio technology (5G, LTE or 3G), the channel indicators RSRP, RSRQ, SNR, CQI and RSSI (plus RSRP/RSRQ of the 5G NR carrier) and the downlink throughput. The 5G network is non-standalone (NSA): 5G radio anchored on a 4G core, the architecture deployed by operators at the time.

B. Which traces measure the network?

The dataset has three applications (Table I). In the two video-streaming applications, the measured throughput is mostly set by the video player, not by the network: once its buffer is full, the player stops downloading, and 74 % of Netflix seconds show zero throughput. Forecasting these values would mean forecasting the player. We therefore keep the 21 *file-download* traces, during which the phone downloads continuously and the measured throughput approximates the capacity offered by the network (only 3.8 % zero seconds).

TABLE I
APPLICATIONS IN THE DATASET

Application	Traces	Samples	Zero thr.	Median (Mbit/s)
File download (kept)	21	43,208	3.8 %	12.1
Netflix	33	72,832	73.7 %	0.0
Amazon Prime	29	72,671	37.6 %	0.3

TABLE II
FILE-DOWNLOAD TRACES AFTER CLEANING

	Traces	Seconds	Thr. (Mbit/s)		5G	Cell
			median	p90	share	changes
Driving	16	28,551	9.0	96.8	51 %	686
Static	5	17,217	36.1	205.0	100 %	0

C. Cleaning

Each trace is put on a regular 1-second grid: rows sharing a timestamp are averaged, and gaps of up to 3 s are filled forward (14.9 % of grid seconds). Filled seconds are used to compute features but never as evaluation targets (Section E). Values outside the 3GPP reporting ranges, such as Android's "unavailable" code $2^{31}-1$ or RSRP $= -200$ dBm, are set to missing. The result has 45,768 seconds (Table II). Driving and static conditions differ strongly (Fig. 2): the static sites were all served by 5G, with a median of 36 Mbit/s, while driving combines 5G, LTE and 3G with a median of 9 Mbit/s and 686 cell changes. Fig. 1 shows the routes and how throughput varies along them.

III. METHOD

A. Problem formulation

For trace i and second t, let r_t be the measured downlink throughput. Given the features $\boldsymbol{x}_t$ available at time t, we forecast the *mean throughput over the next h seconds*:

$$y_{t,h} = \frac{1}{h}\sum_{k=1}^{h} r_{t+k}, \qquad h \in \{1,3,5\}\ \text{s}. \tag{1}$$

The mean over a window, rather than the value at $t+h$, is what an application needs: a video player downloading a chunk during the next seconds is limited by the throughput averaged over that time. Fig. 3 summarises the method.

B. Features

1) History: Lags $r_t, \ldots, r_{t-4}$; rolling mean and standard deviation over 3, 5, 10 and 30 s; minimum and maximum over 5 s; an exponentially weighted mean ($\alpha = 0.3$); and the *harmonic mean* of the last 5 s,

$$\bar{r}_t^{\text{HM}} = \frac{5}{\sum_{k=0}^{4} 1/r_{t-k}}, \tag{2}$$

the estimator used by classic video players [6] because it is dominated by low values, which makes it robust to short peaks but pessimistic.

2) Radio: RSRP, RSRQ, SNR, CQI, RSSI and the NR-specific RSRP/RSRQ, their change over the last 3 s, and 5-s means of SNR and RSRP. They describe the channel quality, from which the achievable rate depends (Shannon: $C = B\log_2(1+\text{SNR})$).

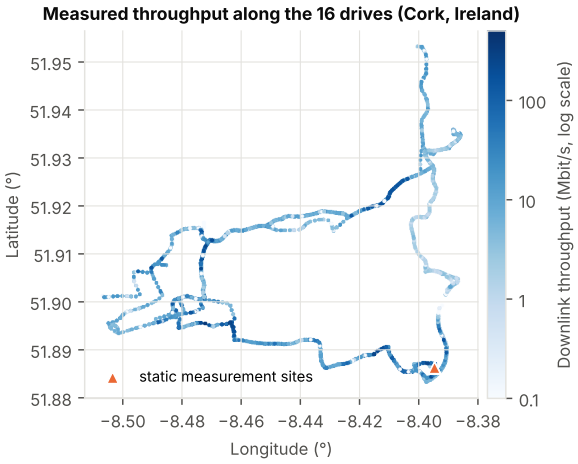


Fig. 1. Routes of the 16 drives, coloured by measured throughput (log scale), and the five static sites. Throughput varies along the routes, which motivates a spatial view.

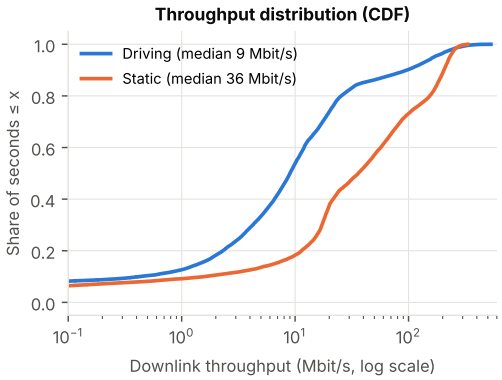


Fig. 2. Cumulative distribution of throughput. The distribution is heavy-tailed (0 to 533 Mbit/s) and four times higher when static than when driving.

3) Mobility: Speed v, acceleration, radio technology, the number of cell changes in the last 5 and 30 s, the time since the last cell change, and the maximum *Doppler shift*

$$f_D = \frac{v}{c} f_c, \tag{3}$$

with c the speed of light and f_c the carrier frequency (3.5 GHz for 5G, the n78 band used in Ireland; 1.8 GHz for LTE; an assumption, as the dataset does not log the band). Doppler sets how fast the channel changes: its coherence time is approximately $T_c \approx 0.423/f_D$ [7]. At the maximum speed of the dataset (89 km/h), $f_D = 288$ Hz and $T_c \approx 1.5$ ms.

4) Space: Latitude and longitude, and a *space-time throughput prior* (Fig. 4). The map is divided into cells of about 110×110 m. For each cell, the prior is the median throughput measured there by the *training* drives (cells with fewer than 5 samples fall back to the global median). It is read at the current position $\boldsymbol{p}_t$ and at the position the vehicle will reach in h seconds, extrapolated from its recent motion:

$$\hat{\boldsymbol{p}}_{t+h} = \boldsymbol{p}_t + h\,\hat{\boldsymbol{v}}_t, \qquad \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{p}_t - \boldsymbol{p}_{t-3}}{3}. \tag{4}$$

The question asked to the data is: "what throughput do users usually get where I will be in h seconds?"

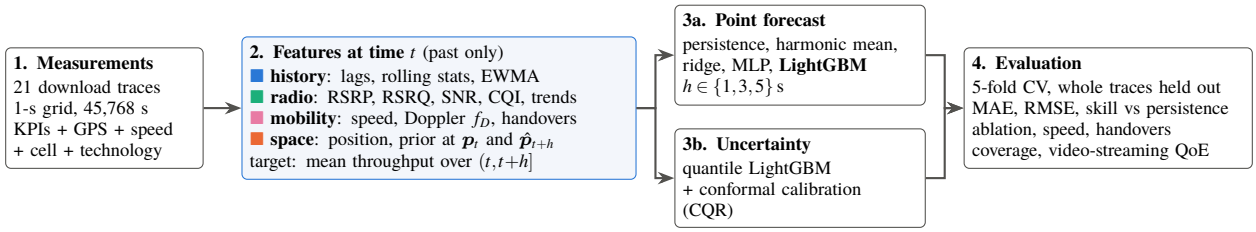


Fig. 3. Forecasting pipeline. All features at time t use only data measured up to t; the space-time prior is learned inside each fold from the training drives only.

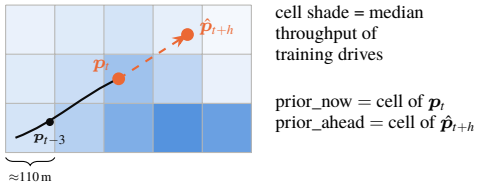


Fig. 4. Space-time prior. The position in h seconds is extrapolated from the motion of the last 3 s (Eq. 4), and the prior is the median throughput that other drives measured in that grid cell. Shades are illustrative.

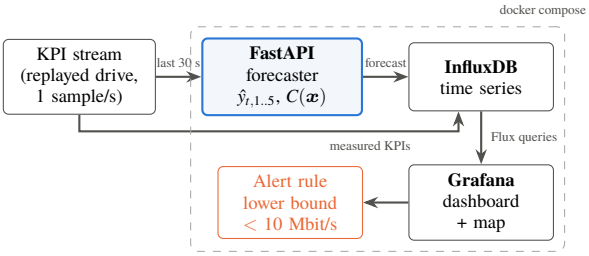


Fig. 5. Real-time architecture. A replayer streams a recorded drive as if it were live; the API returns forecasts and intervals; Grafana plots measured and forecast throughput and raises an alert when a QoS drop is predicted.

C. Models

- **Persistence**: $\hat{y}_{t,h} = r_t$. Hard to beat at short horizons because throughput is strongly autocorrelated (correlation 0.89 between r_t and $y_{t,1}$).
- **Harmonic mean**: $\hat{y}_{t,h} = \bar{r}_t^{\mathrm{HM}}$, the video-player estimator.
- **Ridge regression** on standardised features (median imputation).
- **MLP**: two hidden layers (128, 64), trained on $\log(1+y)$ because of the heavy tail, with early stopping.
- **LightGBM** [8]: gradient-boosted trees (600 trees, 63 leaves, learning rate 0.03, row and column subsampling 0.8). Tree ensembles remain the strongest models on tabular data [9], handle missing values natively and need no scaling.

D. Calibrated prediction intervals

A point forecast says nothing about risk. Two LightGBM models trained with the quantile (pinball) loss estimate the 10 % and 90 % quantiles $\hat{q}_{lo}(\boldsymbol{x})$ and $\hat{q}_{hi}(\boldsymbol{x})$. Such intervals are often too narrow on new data, so we calibrate them with *conformalised quantile regression* (CQR) [10]. On a calibration set of n samples from drives not used to fit the quantile models, we compute the scores

$$E_j = \max\left(\hat{q}_{lo}(\boldsymbol{x}_j) - y_j,\; y_j - \hat{q}_{hi}(\boldsymbol{x}_j)\right), \tag{5}$$

take Q as their $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile, and return

$$C(\boldsymbol{x}) = \left[\max(0,\; \hat{q}_{lo}(\boldsymbol{x}) - Q),\; \hat{q}_{hi}(\boldsymbol{x}) + Q\right]. \tag{6}$$

If calibration and test samples are exchangeable, $P(y \in C(\boldsymbol{x})) \geq 1-\alpha$, here 80 % [11, 12]. Drives are not perfectly exchangeable (different days and routes), so coverage on held-out drives has to be measured, not assumed.

E. Evaluation protocol

1) No leakage: Consecutive seconds of a trace are almost identical, so a random split of samples would put near-duplicates in training and test sets and inflate the scores [13]. We use 5-fold cross-validation in which *whole traces* are held out (stratified so that every fold contains driving and static traces). Every result is computed on drives never seen in training. The spatial prior is re-learned inside each fold from the training drives only. A unit test checks that changing future data never changes past features.

2) Validity mask: A target is scored only if second t and the whole window $(t, t+h]$ were measured, not filled. This leaves 32,531, 20,880 and 11,092 test samples for $h = 1, 3, 5$ s.

3) Metrics: Mean absolute error (MAE), root mean squared error (RMSE), and the *skill* against persistence, $1 - \mathrm{MAE}_{\mathrm{model}}/\mathrm{MAE}_{\mathrm{persistence}}$. For intervals: empirical coverage and mean width.

F. Video-streaming use case

To measure practical value, we replay each trace in a trace-driven adaptive-bitrate (ABR) simulator. A player downloads 2-s chunks over the measured throughput, chooses each chunk's bitrate from the ladder $\{1, 2.5, 5, 8, 16, 25, 40\}$ Mbit/s (HD to 4K) and has a 20-s buffer. Before each chunk, it picks the highest bitrate below its throughput estimate: $0.9\times$ the harmonic mean (the classic rule), $0.9\times$ the LightGBM 3-s forecast, or the CQR lower bound (a risk-aware rule). Quality of experience follows Yin *et al.* [6]:

$$\mathrm{QoE} = \sum_k R_k - \mu \sum_k T_k^{\mathrm{stall}} - \sum_k |R_{k+1} - R_k|, \tag{7}$$

with R_k the bitrate of chunk k, T_k^{stall} the stall time it caused and $\mu = 40$ (one second of stall costs as much as one second at the top quality). The predictions used are the out-of-fold ones: each trace is streamed with models that never saw it.

IV. Real-Time Monitoring System

A forecast is only useful if it reaches the application in time. We therefore deploy the model as a small service (Fig. 5). The **forecasting API** (FastAPI) receives the last 30 s of measurements and computes the features with the same code as offline, which removes any training/serving mismatch. It returns the 1-, 3- and 5-s forecasts and the 80 % interval for 3 s. A **replayer** streams a held-out drive second by second, as a phone or a network probe would, writes the measured KPIs to **InfluxDB** and the forecasts time-stamped at $t+h$, so that forecast and measurement line up when the future arrives.

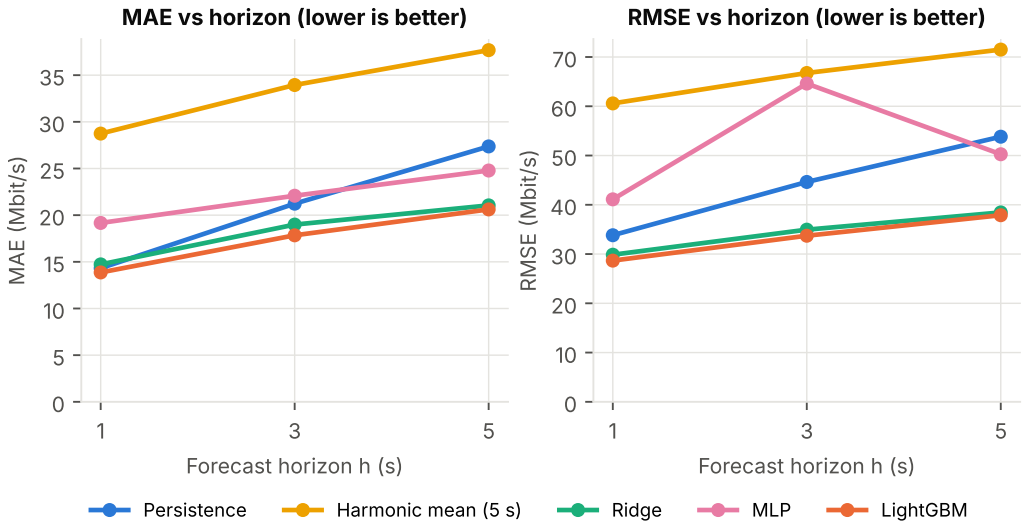


Fig. 6. Error versus forecast horizon. All learned models gain on persistence as the horizon grows; LightGBM is the best everywhere.

TABLE III
FORECAST ERROR ON HELD-OUT DRIVES (MBIT/S)

	$h = 1$ s		$h = 3$ s		$h = 5$ s	
Model	MAE	RMSE	MAE	RMSE	MAE	RMSE
Persistence	14.32	33.82	21.23	44.63	27.37	53.82
Harmonic mean	28.74	60.58	33.94	66.75	37.68	71.51
Ridge	14.73	29.86	18.99	34.95	21.05	38.45
MLP	19.17	41.11	22.08	64.61	24.78	50.26
LightGBM	**13.86**	**28.67**	**17.84**	**33.72**	**20.62**	**37.88**
Skill vs pers.	3 %	15 %	16 %	25 %	25 %	30 %

A provisioned **Grafana** dashboard shows measured vs forecast throughput with the interval band, a map of the vehicle coloured by throughput, the radio KPIs, speed and serving cell. A Grafana alert rule fires when the *lower bound* of the 3-s forecast falls below the rate the service needs (10 Mbit/s for 4K video). Using the lower bound rather than the point forecast means the alert fires when a drop is *plausible*, not only when it is the most likely outcome. The whole stack starts with one `docker compose up`. The deployable model is trained on all traces except the replayed demo drive, so the live demo shows predictions on unseen data.

V. RESULTS

A. Accuracy (Q1)

Table III and Fig. 6 compare the models. LightGBM is the best model at every horizon and beats persistence in **all 15 fold-horizon pairs**. Its advantage grows with the horizon: MAE skill of 3 %, 16 % and 25 % at 1, 3 and 5 s, and RMSE skill of 15 %, 25 % and 30 %. The RMSE gain is larger because the model avoids the large errors persistence makes when throughput jumps. At 1 s, persistence is already hard to beat in MAE: the throughput of the next second is mostly the current one. Ridge regression is close to LightGBM at 5 s, a sign that most of the useful signal is captured by smooth functions of the history. The MLP is unstable across folds. The harmonic mean is by far the worst point forecast: it is biased towards low values by design, a property that is useful for avoiding stalls but not for accuracy.

Static traces are easier to improve on than drives (Table IV): at 3 s, LightGBM cuts the MAE by 23 % when static but by 8 % when driving, where throughput is lower and changes are driven by handovers and technology switches. Fig. 7 shows a 5-minute excerpt of a held-out drive. Persistence follows the measured throughput with a delay; LightGBM anticipates part of the recoveries but also overshoots at times, because a squared-error model predicts the conditional *mean* of a skewed distribution.

TABLE IV
ERROR BY CONDITION AT $h = 3$ S (MBIT/S)

Condition	Model	MAE	RMSE	n
Driving	Persistence	15.36	37.67	13,606
	LightGBM	**14.17**	**30.75**	
Static	Persistence	32.22	55.36	7,274
	LightGBM	**24.72**	**38.66**	

B. What helps beyond history (Q2)

1) Ablation: Fig. 8 adds the feature groups one at a time. Over all test seconds, **history alone is the best** (MAE 17.25); radio, mobility and space each add a little error (17.51, 17.64, 17.84), a sign of mild overfitting with only 16 or 17 training drives per fold. The picture changes right after a handover (1,447 seconds within 5 s of a cell change): the history-only model is *worse than persistence* (15.27 vs 15.03), because its features still describe the previous cell, while adding radio and mobility features brings the error to 14.60, 3 % better than persistence. Context features matter exactly where the past stops being informative. SHAP values [14] on the deployed model (Fig. 9) agree: history accounts for 84 % of the total attribution, radio 8 %, space 5 % and mobility 3 %.

2) Space: The space-time prior is the most important non-history feature (Fig. 9), but it does not improve accuracy on held-out drives. This is not a coverage problem: 69 to 99 % of test points fall in a grid cell with a learned prior. The likely reason is that throughput at a given place depends more on *when* one passes (cell load, time of day, whether the phone is on 5G or LTE) than on where: with 16 drives over ten weeks, the median of past passes is a noisy estimate of the next one.

3) Speed and Doppler: If faster movement made the channel harder to predict, error would grow with speed. It does not (Fig. 10, left): the normalised MAE stays between 0.42

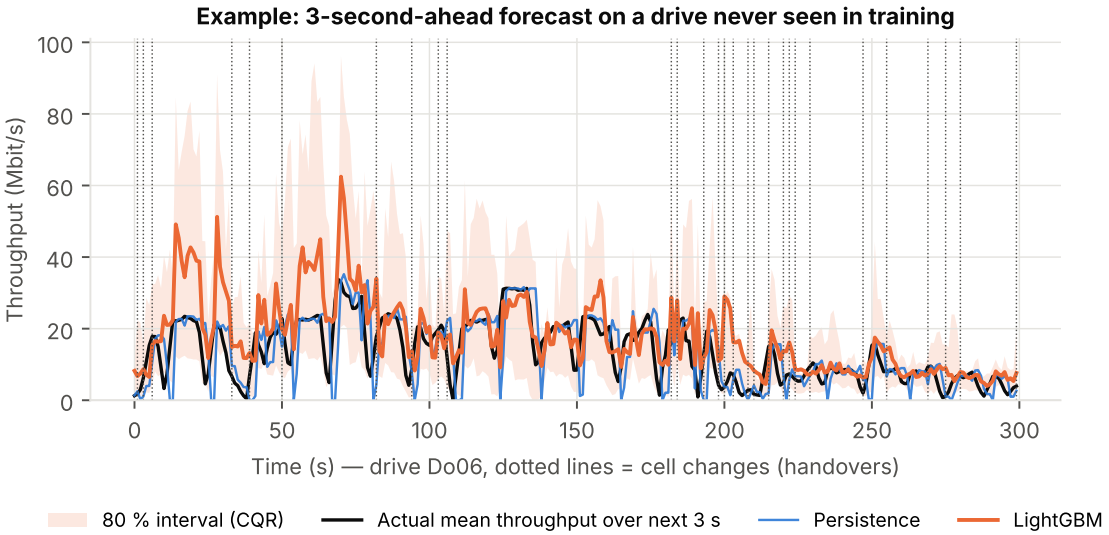


Fig. 7. Five minutes of a drive never seen in training: actual mean throughput over the next 3 s, persistence, LightGBM and the CQR 80 % interval. Dotted lines mark cell changes.

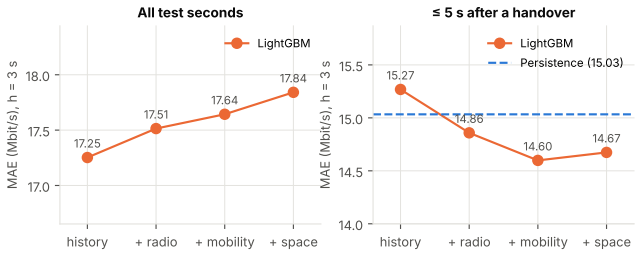


Fig. 8. Cumulative ablation at $h = 3$ s. Left: all test seconds, history alone is best. Right: within 5 s after a handover, radio and mobility features are needed to beat persistence.

TABLE V
80 % PREDICTION INTERVALS, $h = 3$ s

Method	Coverage	Range over folds	Width (Mbit/s)
Quantile regression	72.0 %	69.1–75.6 %	46.7
CQR (calibrated)	**78.0 %**	75.0–80.6 %	47.9

and 0.46 for LightGBM, from walking pace to over 45 km/h. Equation (3) explains why: at these speeds, f_D stays below 300 Hz and the coherence time is between 1.5 and a few milliseconds. A 1-s measurement averages hundreds of coherence intervals, so the fast fading that Doppler controls is invisible at this time scale. What does affect predictability is large-scale change, i.e. handovers (Fig. 10, right): the error peaks 3 to 10 s after a cell change and is lowest far from it.

C. Uncertainty (Q3)

Uncalibrated quantile regression covers only 72.0 % of held-out targets with its nominal 80 % intervals (Table V). Conformal calibration raises coverage to 78.0 % (75.0 to 80.6 % per fold) for a 3 % wider interval. The remaining 2-point gap is the price of non-exchangeable drives: calibration drives differ from test drives. Intervals adapt to the situation (Fig. 11): their median width is about 5 Mbit/s when a low throughput is forecast and above 100 Mbit/s when it exceeds 50 Mbit/s, which matches the larger variability of high-throughput 5G.

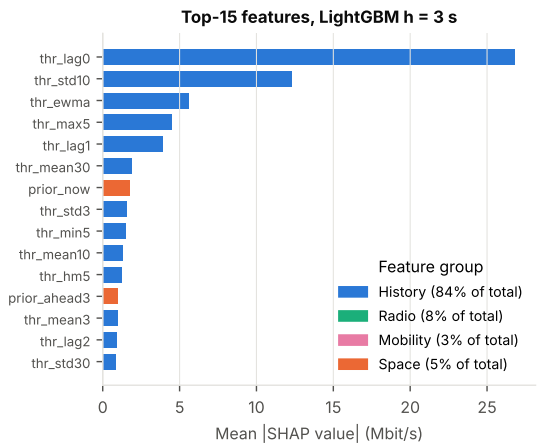


Fig. 9. SHAP feature importance of LightGBM ($h = 3$ s). The current throughput and its recent variability dominate; the space prior is the first non-history feature.

D. Video streaming (Q3)

Fig. 12 reports the streaming simulation over the 21 traces. With the LightGBM forecast, the player streams at 18.2 Mbit/s on average instead of 11.1 Mbit/s with the harmonic mean, switches quality less often (8.6 vs 11.0 per minute), at the cost of more stalls (2.8 % vs 1.4 % of playback). The QoE of Eq. (7) rises from 5.6 to 13.3 per chunk, and is higher on **19 of the 21 traces**. Using the CQR lower bound instead gives the fewest stalls (1.2 %) at the same mean bitrate as the harmonic mean, and a higher QoE on 14 of 21 traces. The interval therefore offers a direct trade-off: the point forecast maximises quality, while the lower bound is the safe choice for stall-sensitive services.

VI. DISCUSSION

1) Beating persistence is the real test: Persistence is a strong baseline at 1 s and many published accuracies would look modest against it. Reporting skill against persistence, on drives held out entirely, is what makes the gains of 16 to 25 % at 3 and 5 s credible.

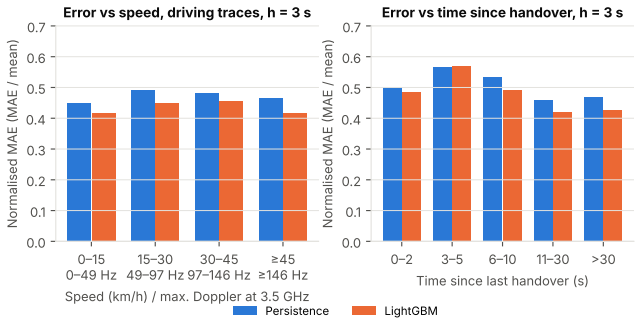


Fig. 10. Normalised MAE (MAE divided by mean throughput) on driving traces at $h = 3$ s. Left: error does not grow with speed or Doppler. Right: error peaks a few seconds after a handover.

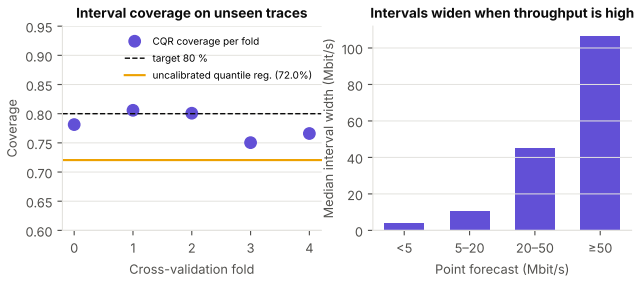


Fig. 11. Left: CQR coverage per fold against the 80 % target and the uncalibrated quantile regression. Right: interval width grows with the forecast level.

2) Space-time information: a negative result: The idea that a vehicle can anticipate QoS from where it will be is appealing, and the prior is the most used non-history feature, yet it does not reduce error here. Two conditions would likely be needed: many more passes per location (so that the prior captures the location effect rather than one day's load) and longer horizons, where the current throughput says less about the future. Datasets from vehicle fleets, such as Berlin V2X [15], are the natural next test.

3) Physics sets the right time scale: The absence of a speed effect is consistent with channel physics: Doppler governs millisecond-scale fading, which 1-s KPIs average out. At the 1-to-5-s scale, mobility acts through handovers and technology changes, and that is where context features help.

4) Limitations: One operator, one city and 21 download traces, collected in 2019–2020 on NSA 5G; the carrier frequencies used for Doppler are assumed; the KPIs come from a phone application at about 1 Hz; hyperparameters were not tuned; the streaming simulator ignores TCP dynamics; the real-time system was tested with a replayed drive, not inside an operator network.

VII. Conclusion

On real 5G measurements and drives never seen in training, a gradient-boosted model forecasts the throughput of the next 1 to 5 s better than persistence in every fold, with gains growing with the horizon (up to 25 % MAE and 30 % RMSE at 5 s). History carries most of the signal; radio and mobility indicators matter right after handovers; and a space-time prior learned from other drives does not help on this dataset. Conformal calibration brings the coverage of 80 % intervals from 72 to 78 %, and in a streaming simulation the forecast more than doubles the quality of experience of the classic estimator. The model runs as a real-time service with a Grafana dashboard and predictive alerts. Future work: fleet-scale data for the spatial prior, longer horizons, 5G standalone networks, and online re-calibration of the intervals.

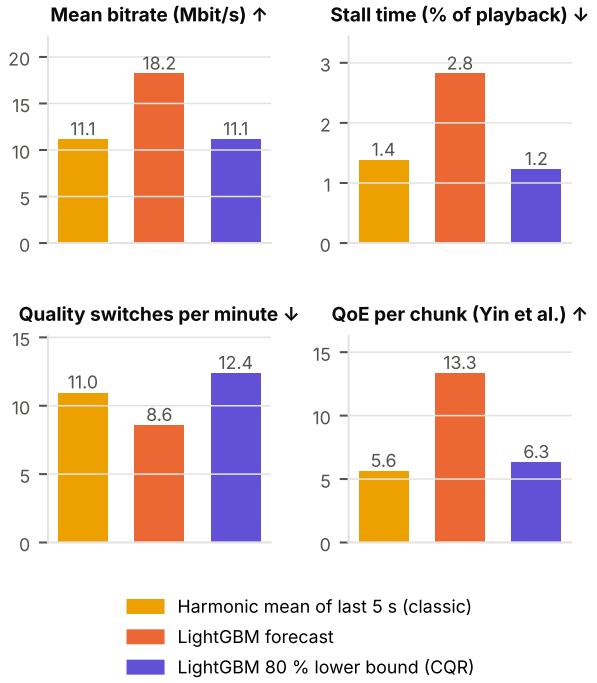


Fig. 12. Trace-driven video streaming (mean over 21 traces, out-of-fold predictions). The forecast raises bitrate and QoE; its calibrated lower bound minimises stalls.

References

[1] 5GAA Automotive Association, "Making 5G proactive and predictive for the automotive industry," White Paper, Jan. 2020.

[2] 3GPP, "Architecture enhancements for 5G System (5GS) to support network data analytics services," TS 23.288.

[3] C. Yue, R. Jin, K. Suh, Y. Qin, B. Wang, and W. Wei, "LinkForecast: Cellular link bandwidth prediction in LTE networks," *IEEE Trans. Mobile Comput.*, vol. 17, no. 7, pp. 1582–1594, 2018.

[4] A. Narayanan *et al.*, "Lumos5G: Mapping and predicting commercial mmWave 5G throughput," in *Proc. ACM IMC*, 2020, pp. 176–193.

[5] D. Raca, D. Leahy, C. J. Sreenan, and J. J. Quinlan, "Beyond throughput, the next generation: A 5G dataset with channel and context metrics," in *Proc. ACM MMSys*, 2020, pp. 303–308.

[6] X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, "A control-theoretic approach for dynamic adaptive video streaming over HTTP," in *Proc. ACM SIGCOMM*, 2015, pp. 325–338.

[7] T. S. Rappaport, *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2002.

[8] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.

[9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. NeurIPS Datasets and Benchmarks*, 2022.

[10] Y. Romano, E. Patterson, and E. Candès, "Conformalized quantile regression," in *Proc. NeurIPS*, 2019.

[11] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[12] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv:2107.07511, 2021.

[13] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Security Symp.*, 2022.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017.

[15] R. Hernangómez *et al.*, "Berlin V2X: A machine learning dataset from multiple vehicles and radio access technologies," in *Proc. IEEE VTC2023-Spring*, 2023.

A. CODE – METHOD MAPPING

File	Role
`src/qosf/data.py`	Download, 1-s grid, cleaning (§II)
`src/qosf/features.py`	Features, target (1), Doppler (3), prior (4)
`src/qosf/models.py`	Baselines, ridge, MLP, LightGBM
`src/qosf/conformal.py`	CQR, Eq. (6)
`src/qosf/abr.py`	Streaming simulator, QoE (7)
`scripts/run_experiments.py`	All experiments of §V
`scripts/train_final.py`	Deployable model
`realtime/`	API, replayer, Grafana (§IV)
`tests/`	Leakage, target, Doppler, ABR, API tests

Reproduction (from the repository root):

- `pip install -r requirements.txt`
- `python scripts/run_experiments.py`
- `python scripts/train_final.py`
- `python scripts/make_figures.py`
- live demo: `docker compose up --build`, then `http://localhost:3000`